

Minimum Cost to Merge Stones (LC 1000)

© Authored by: Arpit Raj, LNMIIT Jaipur

Date: August 28, 2026

Amazon

DE Shaw

VMware

1. Problem Understanding & Constraints

- Given n piles of stone.
- 1 move means merging k consecutive piles into one single pile.
- **Cost of move:** total number of stones in these k piles.

2. Working Examples

Example 1

Input: Array: [3, 2, 4, 1], $k = 2$

- Step 1: Merge [3, 2] $\rightarrow$ [5]. Array becomes [5, 4, 1]
- Step 2: Merge [4, 1] $\rightarrow$ [5]. Array becomes [5, 5]
- Step 3: Merge [5, 5] $\rightarrow$ [10]. Array becomes [10]

Total Cost: $(3 + 2) + (4 + 1) + (5 + 5) = 20$

Example 2

Input: Array: [3, 2, 4, 1], $k = 3$

- Step 1: Merge [3, 2, 4] $\rightarrow$ [9]. Array becomes [9, 1]

- Can't get 1 pile at end, hence return -1.

3. Feasibility Condition (Base Check)

We need to go from n piles to 1 pile. Each merge reduces the number of piles by $k - 1$ (since k piles are removed and 1 is added).

```
if (((n - 1) % (k - 1)) != 0) return -1;
```

4. DP Intuition & Approach

Let's define: $dp[l][r]$ = min cost to merge stones $[l \dots r]$

Interval Merging Strategy

Now while merging $[l \dots r]$, we will have 2 parts:



These 2 parts will be merged.

Also, we will process smaller intervals first. First 1-sized, then 2-sized, and so on because bigger intervals are made from smaller ones.

- **Base Case:** $dp[i][i] = 0$ (No cost as no merge is needed for a single element).

Transition Implementation

```
for (int j = 1; j < r; j += k-1) { int curr = dp[l][j] + dp[j+1][r]; dp[l][r] = min(dp[l][r], curr); }
```

Notes from Explanation:

1 merge k piles ko 1 pile banata, isiliye move by $(k-1)$.

"Ye to ho gaya bacho ka cost, ab parent cost hoga jisko jod rhe ho unka sum."

```
 $dp[l][r] += pref[r+1] - pref[l]$ 
```

Last me return $dp[0][n-1]$.

5. C++ Code

```
class Solution {
public:
    int mergeStones(vector<int>& stones, int k) {

        int n=stones.size();
        if(((n-1)%(k-1))!=0) return -1;

        vector<int> pref(n+1,0);
        for(int i=0;i<n;i++){
            pref[i+1]=pref[i]+stones[i];
        }

        vector<vector<int>> dp(n,vector<int>(n,INT_MAX));
        for(int i=0;i<n;i++){
            dp[i][i]=0;
        }

        for(int len=2;len<=n;len++){
            for(int l=0;l+len-1<n;l++){
                int r=l+len-1;

                //[l....j][j+1.....r]

                for(int j=l;j<r;j+=k-1){
                    int curr=dp[l][j]+dp[j+1][r];
                    dp[l][r]=min(dp[l][r],curr);
                }

                if(((len-1)%(k-1))!=0) continue;
                dp[l][r]+=pref[r+1]-pref[l];
            }
        }

        return dp[0][n-1];
    }
};
```